EECS 678 Extra review questions – Midterm1 Fall 2026

1 Chapter 1

1. Explain the User's and System's view of the operating system.
2. Explain the operating system goals.
3. What is a multiprogramming OS? How does it differ from a batch OS? How does it differ from a time-sharing OS?
4. How does an OS ensure protection between two user processes?
5. How does an OS ensure its own protection as well as that of the other system resources?
6. Sort the following levels of storage hierarchy by their access time and size.
   (a) Registers, (b) cache, (c) disk
7. What factors restrict the number of architected registers?
8. What factors restrict the size of the cache? Why have multiple levels of caches?
9. Explain if true or false:
   (a) The architecture registers are managed by the hardware.
   (b) The user can directly manage the disk.
10. The OS provides a virtualization. Explain.

2 Chapter 2

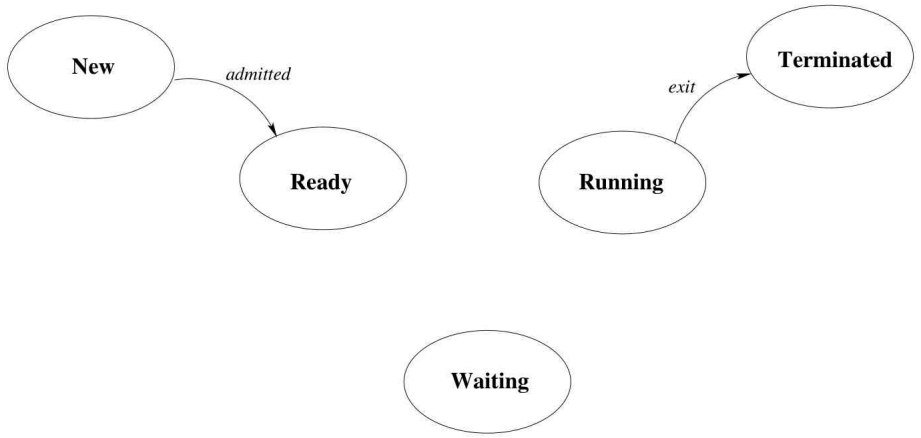
1. Name a few OS services to help the user.
2. Name a few OS services for efficient system operation.
3. What are system calls? How do they differ from other function calls?
4. What is an API? How is an API call different from a system call? Which one would you prefer to use in your high-level program?
5. What are the limitations of passing function arguments in registers? How are these limitations overcome?
6. What do the following system tools do: (a) compiler, (b) assembler, (c) linker, and (d) loader?

3 Chapter 3

1. Define process. How is a process different from a program?

2. Illustrate (draw) the process address space. What component of your high-level C program goes into each section of the process address space?

3. The process state diagram below shows all the possible process states, but only two process transitions. Complete the figure by illustrating how the process changes states on the following events: (a) interrupt, (b) scheduler dispatch, (c) I/O or event completion, and (d) I/0 or event wait.



4. Which of the following typical kinds of CPU instructions should be privileged? Explain your reasoning briefly.
   (a) Set value for hardware timer
   (b) Read hardware clock
   (c) Set a value in the shared memory region.
   (d) Switch from user to OS execution mode
   (e) Turn off hardware interrupts.

5. Describe what a kernel must do to switch context between two processes. What are the benefits and drawbacks of context switching for an OS?

6. What is the purpose of the fork and exec system calls?

7. Using appropriate system calls, have the program below create a new process, and execute the program /bin/ps. It is not important to get all the function arguments and their positions correct.

```
int main()
{
    pid_t pid;

    /*create a new process*/
    pid =
    if(pid == 0){
```

```
    }
    else if(pid < 0){


    }
    else if (pid > 0){


    }
}
```

8. Compare the shared memory and message passing IPC models as regards: (a) characteristics (b) efficiency (c) ease of use. Explain

9. Consider the program below:

```
int main()
{
    pid_t pid;
    int shared_num = 0;

    /* create a pipe */
    ...

    /* create a process */
    pid = fork();
    if(pid > 0){
        /* parent process */
        /* write numbers to the shared variable*/
        int i;
        for(i=0 ; i<10000 ; i++)
            shared_num += 1;

        wait();
        fprintf(stdout, "Shared variable = %d\n", shared_num);
    }
    else{
        /* child process */
        int i;
        for(i=0 ; i<10000 ; i++)
            shared_num -= 1;
    }
}
```

Use pipes to synchronize access to the shared variable so that the final result printed in the parent process is 0. (You need to first understand why the answer printed may be different from 0).

10. Compare shared memory based IPC with message passing based IPC.
11. What are the advantages and limitations of pipes IPC? How do some other IPC mechanisms address these drawbacks?
12. Explain the (a) server and (b) client side sequence of calls to set a socket between a client and a server.

4 Chapter 4

1. Draw the process address space of a process with two threads. Indicate the stack, heap, text, and data space of the two threads. How is the process address space different if we had two processes instead of two threads?
2. What the (a) advantages and (b) disadvantages of threads over processes?
3. Differentiate between (a) user-level and (b) kernel-level threads. What are their advantages and disadvantages over each other?
4. Name and illustrate the three multithreading models.
5. What is the main disadvantage of the many-to-one multithreading model?
6. What system call is used to create threads in Linux?
7. Write a simple program that contains two global variables num1 and num2. The program then: (a) creates two threads, (b) Thread-1 performs num1+num2 and returns the result, (c) Thread-2 performs num1-num2 and returns the result, (d) Main thread displays the two returned results and exits.
8. How do system calls fork and exec operate if invoked from multi-threaded programs? (You may use Linux as an example.)
9. Explain the two mechanisms of thread cancellation.
10. How are synchronous and asynchronous signals handled in a multi-threaded program?